

(6) Handling Text

Compute Word Frequency overall

```
# your code goes here
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
import numpy as np

vectorizer = CountVectorizer(
    min_df=1,
    token_pattern=r"\b[a-z]+\b"
)

X = vectorizer.fit_transform(df_data["line"])

corpus_freq = np.asarray(X.sum(axis=0)).ravel()
terms = vectorizer.get_feature_names_out()

freq_df = pd.DataFrame({
    "term": terms,
    "corpus_frequency": corpus_freq
})

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.hist(freq_df["corpus_frequency"], bins=np.logspace(0,6,100))
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Corpus frequency (log scale)")
plt.ylabel("Number of terms (log scale)")
plt.title("Distribution of corpus term frequencies")
plt.show()
```

TF-IDF from zero construction example (probably need to change the way tokens are gotten)

```
import numpy as np
import pandas as pd
from itertools import chain
```

```

# -----
# Load data
# -----
data_path = "data/exam3.jsonl"
df = pd.read_json(data_path, lines=True)

# -----
# Filter Chandler Bing utterances
# -----
chandler_df = df.loc[df["speaker"] == "Chandler Bing", ["tokens",
"episode"]]

# -----
# Map each episode to a row index (236 episodes total)
# -----
episodes = sorted(chandler_df["episode"].unique())
episode_to_index = {ep: i for i, ep in enumerate(episodes)}

# -----
# Build vocabulary L: all distinct tokens uttered by Chandler
# -----
unique_tokens = set()

for tokens in chandler_df["tokens"]:
    unique_tokens.update(chain.from_iterable(tokens))

vocab = sorted(unique_tokens)

print("First 10 tokens:", vocab[:10])
print("Last 10 tokens:", vocab[-10:])

# -----
# Build TF matrix X (episodes x vocabulary)
# -----
num_episodes = len(episodes)
num_tokens = len(vocab)

X = np.zeros((num_episodes, num_tokens))

word_to_index = {word: j for j, word in enumerate(vocab)}

for _, row in chandler_df.iterrows():
    i = episode_to_index[row["episode"]]
    for token in chain.from_iterable(row["tokens"]):
        j = word_to_index[token]
        X[i, j] += 1

print("TF matrix shape:", X.shape)

```

```

# Chandler saying "joey" in first episode
joey_id = word_to_index["joey"]
print("TF('joey', episode 1):", X[0, joey_id])

# -----
# Build TF-IDF matrix T
# -----

TF = X

# Document Frequency: number of episodes containing each word
DF = np.count_nonzero(TF > 0, axis=0)

# Inverse Document Frequency (exam formula)
IDF = np.log(num_episodes / DF)

# TF-IDF matrix
T = TF * IDF

print("TF-IDF matrix shape:", T.shape)
print("TF-IDF('joey', episode 1):", T[0, joey_id])

```

2024 Another example of building a TF-IDF but using the existing functionalities and also some other sorting magic

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

# -----
# 1. Build one document per (negotiation_id, agent)
# -----

# Combine all messages from the same agent in the same negotiation
df_document = (
    df_negotiations
        .groupby(["negotiation_id", "agent"])["message"]
        .apply(lambda msgs: "\n".join(msgs))
        .reset_index()
        .rename(columns={"message": "document"})
)

# Attach metadata (including outcome)
df_document = pd.merge(
    df_document,
    df_meta,
    on=["negotiation_id", "agent"],
    how="inner"
)

```

```

)

print(f"Number of documents: {len(df_document)}")

# -----
# 2. Define outcome thresholds (top / bottom 10%)
# -----

top_threshold = df_document["outcome"].quantile(0.9)
bottom_threshold = df_document["outcome"].quantile(0.1)

print(f"Top 10% outcome threshold: {top_threshold}")
print(f"Bottom 10% outcome threshold: {bottom_threshold}")

# -----
# 3. Convert text to TF-IDF features
# -----

# TF-IDF creates a matrix:
#   rows    = documents
#   columns = words (features)
#   values  = TF-IDF scores
vectorizer = TfidfVectorizer(
    max_features=100,      # keep only 100 most frequent words globally
    stop_words="english"   # remove common English stopwords
)

tfidf_matrix = vectorizer.fit_transform(df_document["document"])

print(f"TF-IDF matrix shape: {tfidf_matrix.shape}")
# Example: (num_documents, 100)

# -----
# 4. Select top and bottom negotiators
# -----

top_indices = df_document[df_document["outcome"] > top_threshold].index
bottom_indices = df_document[df_document["outcome"] <
bottom_threshold].index

# -----
# 5. Compute average TF-IDF scores per group
# -----

# mean(axis=0) → average across documents (rows)
# .A1          → convert 1×N matrix to flat 1D array

```

```

# argsort() → return indices that sort values (low → high)

top_word_order = (
    tfidf_matrix[top_indices]
        .mean(axis=0)
        .A1
        .argsort()
)

bottom_word_order = (
    tfidf_matrix[bottom_indices]
        .mean(axis=0)
        .A1
        .argsort()
)

# -----
# 6. Map word indices back to actual words
# -----

# Vocabulary in the same column order as tfidf_matrix
vocabulary = vectorizer.get_feature_names_out()

# Take the last 10 indices (highest scores)
# Reverse them so the most important word comes first
top_10_terms = [
    vocabulary[i]
    for i in top_word_order[-10:][::-1]
]

bottom_10_terms = [
    vocabulary[i]
    for i in bottom_word_order[-10:][::-1]
]

# -----
# 7. Results
# -----

print(f"Best 10% negotiators – top terms: {top_10_terms}")
print(f"Worst 10% negotiators – top terms: {bottom_10_terms}")

```

DF-ITF and logistic

```

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# -----
# 1. Create binary success label
# -----

# Use the median outcome as the decision boundary
median_outcome = df_document["outcome"].median()
print(f"Median outcome: {median_outcome}")

# success = 1 → above median
# success = 0 → at or below median
df_document["success"] = (df_document["outcome"] >
median_outcome).astype(int)

# -----
# 2. Convert text to TF-IDF features
# -----

# TF-IDF matrix:
#   rows    → documents
#   columns → unigrams + bigrams
#   values  → TF-IDF weights
vectorizer = TfidfVectorizer(
    max_features=100,      # limit vocabulary size
    stop_words="english",  # remove common English words
    ngram_range=(1, 2)    # include unigrams and bigrams
)

X = vectorizer.fit_transform(df_document["document"])
y = df_document["success"]

print(f"TF-IDF matrix shape: {X.shape}")

# -----
# 3. Train / test split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=99
)

```

```

print(f"Training set shape: {X_train.shape}")
print(f"Test set shape: {X_test.shape}")

# -----
# 4. Train logistic regression classifier
# -----

# Logistic regression learns one weight per word feature
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

# -----
# 5. Evaluate model performance
# -----

y_pred = log_reg.predict(X_test)

print(
    classification_report(
        y_test,
        y_pred,
        target_names=["success = 0", "success = 1"]
    )
)

# -----
# 6. Interpret model coefficients (word importance)
# -----

# One coefficient per feature (word / bigram)
coefficients = log_reg.coef_[0]

# Highest positive weights → strongly predict success = 1
top_positive_indices = coefficients.argsort()[-5:][::-1]

# Most negative weights → strongly predict success = 0
top_negative_indices = coefficients.argsort()[:5]

# Map indices back to actual terms
vocabulary = vectorizer.get_feature_names_out()

top_positive_terms = [vocabulary[i] for i in top_positive_indices]
top_negative_terms = [vocabulary[i] for i in top_negative_indices]

print(f"Top 5 terms predicting HIGH outcomes: {top_positive_terms}")

```

```
print(f"Top 5 terms predicting LOW outcomes: {top_negative_terms}")
```